

Build Local LLM

14 June 2026 11:50

I'll check the current small-model and fine-tuning landscape before laying out the pipeline, since both move quickly. "Build a local LLM" needs one honest caveat up front: you don't train a useful model from scratch on a laptop — that takes thousands of GPU-hours. What "build your own local SLM" realistically means is **take a pretrained small base model and make it yours**, either by configuring its behavior or fine-tuning it on your data, then serve that customized model behind FastAPI. Here's the full pipeline with that branch made explicit.

Stage 1 — Pick the base small model

Match the model to your RAM, since an oversized model swaps to disk and crawls. Current sensible picks:

- **8 GB RAM:** Llama 3.2 3B fits in ~2 GB at Q4 and runs 30+ tok/s on an M2 MacBook Air; Phi-4-mini and Gemma 3 4B are also strong here. For coding on 8 GB, Qwen 2.5 7B is a better choice than base Llama.
- **16 GB RAM:** this is the sweet spot — Qwen 3 7B, Gemma 3 12B, or 14B-class models (Qwen 2.5 14B, Phi-4, Gemma 3 12B) reach roughly 80–90% of frontier-model quality on everyday tasks.
- Newer options worth a look if your hardware allows: Gemma 4 E4B runs on about 3 GB VRAM and adds multimodal/audio support, and Qwen 3.5 variants.

Pull whichever you choose through Ollama, e.g. `ollama pull llama3.2:3b`. Ollama is your inference engine — it handles quantization and serves an HTTP API at `localhost:11434`.

Stage 2 — "Build" your model (the customization step)

This is where it becomes *yours*. Two paths, pick by effort:

Path A — Modelfile (minutes, no training). Ollama lets you bake a system prompt, parameters, and template into a named model variant. Create a file called `Modelfile`:

```
FROM llama3.2:3b
```

```
PARAMETER temperature 0.3
```

```
PARAMETER num_ctx 8192
```

```
SYSTEM """You are a support assistant for Exelcius. Answer concisely  
and only from provided context. If unsure, say so."""
```

Then build it: `ollama create my-slm -f Modelfile`. You now have a model called `my-slm`. This covers most "custom local LLM" needs — behavior, tone, defaults.

Path B — Fine-tune on your own data (hours, needs a GPU ideally). When you want the model to actually *learn* your domain, use LoRA/QLoRA (parameter-efficient fine-tuning — you train small adapter layers, not the whole model). The pipeline:

1. Prepare data as instruction pairs (JSONL of `{"instruction", "input", "output"}`).
2. Fine-tune with a tool like Unsloth or HuggingFace PEFT — QLoRA quantizes the base to 4-bit so a 3B model fits in modest VRAM.
3. Merge the adapter and **export to GGUF** format.
4. Import into Ollama with a Modelfile (`FROM ./my-finetuned.gguf`) and `ollama create`.

Path B is a real undertaking; start with Path A unless you have a dataset and a clear reason the model must learn new knowledge.

Stage 3 — Put FastAPI in front

Ollama already serves the model; FastAPI becomes your controlled gateway — your own request/response shape, auth, logging, rate limits, and a place to add RAG later. The key feature to get right is **streaming**, so tokens appear as they generate instead of after a long wait:

```
from fastapi import FastAPI
```

```
from fastapi.responses import StreamingResponse
```

```
from pydantic import BaseModel
```

```

import ollama
app = FastAPI()
MODEL = "my-slm" # the model you built in Stage 2
class Chat(BaseModel):
    message: str
@app.post("/chat")
def chat(req: Chat):
    resp = ollama.chat(model=MODEL,
        messages=[{"role": "user", "content": req.message}])
    return {"reply": resp["message"]["content"]}
@app.post("/stream")
def stream(req: Chat):
    def gen():
        for part in ollama.chat(
            model=MODEL,
            messages=[{"role": "user", "content": req.message}],
            stream=True,
        ):
            yield part["message"]["content"]
    return StreamingResponse(gen(), media_type="text/plain")
@app.get("/health")
def health():
    return {"model": MODEL, "status": "ok"}

```

Stage 4 — Run and verify

ollama serve # if not already running as a service

uvicorn main:app --reload # starts FastAPI on :8000

Test at <http://localhost:8000/docs>. The whole chain — model, inference, API — runs offline on the laptop.

Production notes for the serving layer

A few things that bite people once it's more than a toy:

- **Concurrency.** A single Ollama instance processes requests largely sequentially; a small model on a laptop will struggle under parallel load. Use async def endpoints with the async Ollama client, and don't expect many simultaneous users.
- **Timeouts and context limits.** Long prompts can exceed num_ctx or hang; set request timeouts and cap input length in FastAPI before it reaches the model.
- **Pick the runtime deliberately.** Ollama is the easy path. If you need higher throughput later, llama.cpp's server or vLLM (GPU) serve the same GGUF/weights faster, and your FastAPI layer stays unchanged.

This is also the natural place to reconnect with the RAG work from earlier — the /chat endpoint's prompt is exactly where you'd inject retrieved context, turning this generic SLM server into a document-grounded one.

Want me to combine this with the earlier RAG pipeline into one FastAPI app, or write out the QLoRA fine-tuning script for Path B?